Chapter 4: The Join Operation

Perl

Ví dụ sau minh họa framework DBIx::Class của Perl:

```
my @employees =
   $schema->resultset('Employees')
          ->search({'UPPER(last_name)' => {-like=>'WIN%'}});

foreach my $employee (@employees) {
   # process Employee
   foreach my $sale ($employee->sales) {
      # process Sale for Employee
   }
}
```

DBIx::Class 0.08192 generate các query select N+1:

```
SELECT me.employee_id, me.subsidiary_id
     , me.last_name, me.first_name, me.date_of_birth
  FROM employees me
 WHERE ( UPPER(last_name) LIKE ? )


   SELECT me.sale_id, me.employee_id, me.subsidiary_id
        , me.sale_date, me.eur_value
     FROM sales me
    WHERE ( ( me.employee_id = ?
      AND me.subsidiary_id = ? ) )
```

PHP

Ví dụ Doctrine dùng interface query builder:

```
$qb = $em->createQueryBuilder();
$qb->select('e')
   ->from('Employees', 'e')
   ->where("upper(e.last_name) like :last_name")
   ->setParameter('last_name', 'WIN%');
$r = $qb->getQuery()->getResult();

foreach ($r as $row) {
   // process Employee
   foreach ($row->getSales() as $sale) {
      // process Sale for Employee
   }
}
```

Nested Loops

Doctrine 2.0.5 generate các query select N+1:

```
SELECT e0_.employee_id AS employee_id0 -- MORE COLUMNS
  FROM employees e0_
 WHERE UPPER(e0_.last_name) LIKE ?

   SELECT t0.sale_id AS SALE_ID1 -- MORE COLUMNS
     FROM sales t0
    WHERE t0.subsidiary_id = ?
      AND t0.employee_id = ?

   SELECT t0.sale_id AS SALE_ID1 -- MORE COLUMNS
     FROM sales t0
    WHERE t0.subsidiary_id = ?
      AND t0.employee_id = ?
```

Enabling SQL Logging

Bật SQL logging trong quá trình development và review các SQL statement được generate.

DBIx::Class

X::Class
`export DBIC_TRACE=1` in your shell.

Doctrine

Chỉ thực hiện ở source code level — đừng quên disable setting này trong production. Hãy cân nhắc việc tự xây dựng một configurable logger.

```
$logger = new \Doctrine\DBAL\Logging\EchoSqlLogger;
$config->setSQLLogger($logger);
```

Hibernate (native)

```
<property name="show_sql">true</property> in App.config
hibernate.cfg.xml
```

JPA

Trong persistence.xml, nhưng còn tùy JPA provider:

```
<property name="eclipselink.logging.level" value="FIN
<property name="hibernate.show_sql" value="TRUE"/>
<property name="openjpa.Log" value="SQL=TRACE"/>
```

Hầu hết ORM cũng cung cấp cách programmatic để enable SQL logging. Cách này có rủi ro vô tình deploy setting đó lên production.

Chapter 4: The Join Operation

Dù cách tiếp cận "nested selects" là một anti-pattern, nó vẫn giải thích khá rõ nested loops join. Database thực thi join chính xác như các ORM tool ở trên. Vì vậy, indexing cho nested loops join cũng giống như indexing cho các select statement đã trình bày ở trên. Đó là function-based index trên table EMPLOYEES và concatenated index cho các join predicate trên table SALES:

```
CREATE INDEX emp_up_name ON employees (UPPER(last_name));
CREATE INDEX sales_emp ON sales (subsidiary_id, employee_id);
```

SQL join vẫn hiệu quả hơn cách tiếp cận nested selects — dù thực hiện cùng các index lookup — vì nó tránh được nhiều network communication. Nó thậm chí còn nhanh hơn khi tổng lượng data truyền đi lớn hơn do employee attribute bị lặp lại cho mỗi sale. Đó là vì performance có hai dimension: response time và throughput; trong computer network, chúng ta gọi chúng là latency và bandwidth. Bandwidth chỉ tác động nhỏ đến response time, còn latency có tác động rất lớn. Vì vậy, số database round trip quan trọng đối với response time hơn lượng data được truyền.

>  Tip
>
> Execute join trong database.

Hầu hết ORM tool đều có cách tạo SQL join. Chế độ gọi là eager fetching có lẽ là cách quan trọng nhất. Nó thường được cấu hình ở property level trong entity mapping — ví dụ property employees trong class Sales. Khi đó ORM tool sẽ luôn join table EMPLOYEES khi access table SALES. Cấu hình eager fetching trong entity mapping chỉ hợp lý nếu lúc nào bạn cũng cần employee detail cùng với sales data.

Eager fetching phản tác dụng nếu bạn không cần child record mỗi lần access parent object. Với application dạng telephone directory, load record SALES khi hiển thị employee detail là không hợp lý. Bạn có thể cần sales data liên quan trong những trường hợp khác — nhưng không phải lúc nào cũng cần. Static configuration không giải quyết được vấn đề.

Để đạt performance tối ưu, bạn cần toàn quyền kiểm soát join. Các ví dụ sau cho thấy cách có được mức flexibility cao nhất bằng cách kiểm soát join behavior tại runtime.

Nested Loops

Java

Interface JPA CriteriaBuilder cung cấp method Root<>.fetch() để kiểm soát join. Method này cho phép bạn chỉ định thời điểm và cách join các object được tham chiếu vào main query. Trong ví dụ này, chúng ta dùng left join để lấy tất cả employee, kể cả những employee không có sale.

> Warning
>
> JPA và Hibernate trả về employee cho mỗi sale.
>
> Điều đó có nghĩa một employee có 30 sale sẽ xuất hiện 30 lần. Dù rất khó chịu, đây là behavior được quy định (EJB 3.0 persistency, paragraph 4.4.5.3 “Fetch Joins”). Bạn có thể tự de-duplicate parent relation hoặc dùng function distinct() như trong ví dụ.

```
    queryBuilder.upper(r.get(Employees_.lastName)),
    "WIN%")
);

r.fetch("sales", JoinType.LEFT);
// needed to avoid duplication of Employee records
query.distinct(true);

List<Employees> emp = em.createQuery(query).getResultList();
```

Hibernate 3.6.0 generates the following SQL statement:

```
select distinct
       employees0_.subsidiary_id as subsidiary1_0_0_
       employees0_.employee_id as employee2_0_0
```

Hibernate 3.6.0 generate SQL statement sau:

```
          on employees0_.subsidiary_id=sales1_.subsidiary_id
         and employees0_.employee_id=sales1_.employee_id
 where upper(employees0_.last_name) like ?
```

The query has the expected left join but also an unnecessary `distinct` keyword. Unfortunately, JPA does not provide separate API calls to filter duplicated parent entries without de-duplicating the child records as

Query có left join như kỳ vọng nhưng cũng có keyword distinct không cần thiết. Đáng tiếc là JPA không cung cấp API call riêng để filter các parent entry bị duplicate mà không de-duplicate cả child record

Chapter 4: The Join Operation

tương tự.

Keyword distinct trong SQL query đáng lo vì hầu hết database sẽ thực sự filter các duplicate record. Chỉ một số ít database nhận biết rằng primary key vốn đã bảo đảm tính unique trong trường hợp này.

ıe native Hibernate API solves the problem on the client side using a sult set transformer:

```
Criteria c = session.createCriteria(Employees.class);
c.add(Restrictions.ilike("lastName", 'Win%'));

c.setFetchMode("sales", FetchMode.JOIN);
c.setResultTransformer(Criteria.DISTINCT_ROOT_ENTITY);

List<Employees> result = c.list();
```

Nó generate query sau:

```
select this_.subsidiary_id as subsidiary1_0_1_
     , this_.employee_id as employee2_0_1_
       -- MORE this_ columns on employees
     , sales2_.sale_id as sale1_3_
       -- MORE sales2_ columns on sales
  from employees this_
  left outer join sales sales2_
          on this_.subsidiary_id=sales2_.subsidiary_id
         and this_.employee_id=sales2_.employee_id
 where lower(this_.last_name) like ?
```

Method này tạo ra SQL trực tiếp mà không có clause ngoài ý muốn. Lưu ý rằng Hibernate dùng lower() cho case-insensitive query — một chi tiết quan trọng đối với function-based indexing.

Nested Loops

Perl

Ví dụ sau dùng framework DBIx::Class của Perl:

```
my @employees =
   $schema->resultset('Employees')
          ->search({ 'UPPER(last_name)' => {-like => 'WIN%'}
                   , {prefetch => ['sales']}
                   });
```

DBIx::Class 0.08192 generate SQL statement sau:

```
SELECT me.employee_id, me.subsidiary_id, me.last_name
       -- MORE COLUMNS
  FROM employees me
  LEFT JOIN sales sales
         ON (sales.employee_id   = me.employee_id
        AND  sales.subsidiary_id = me.subsidiary_id)
 WHERE ( UPPER(last_name) LIKE ? )
 ORDER BY sales.employee_id, sales.subsidiary_id
```

Note the `order by` clause—it was not requested by the application. The database has to sort the result set accordingly, and that might take a while.

PHP

Ví dụ sau dùng framework Doctrine của PHP:

```
$qb = $em->createQueryBuilder();
$qb->select('e,s')
   ->from('Employees', 'e')
   ->leftJoin('e.sales', 's')
   ->where("upper(e.last_name) like :last_name")
   ->setParameter('last_name', 'WIN%');
$r = $qb->getQuery()->getResult();
```

Doctrine 2.0.5 generates the following SQL statement:

```
SELECT e0_.employee_id AS employee_id0
       -- MORE COLUMNS
```

Doctrine 2.0.5 generate SQL statement sau:

```
          ON e0_.subsidiary_id = s1_.subsidiary_id
         AND e0_.employee_id = s1_.employee_id
 WHERE UPPER(e0_.last_name) LIKE ?
```


Chapter 4: The Join Operation

```
-----------------------------------------------------------------
|Id |Operation                    | Name        | Rows | Cost |
-----------------------------------------------------------------
| 0 |SELECT STATEMENT             |             |  822 |   38 |
| 1 | NESTED LOOPS OUTER          |             |  822 |   38 |
| 2 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES   |    1 |    4 |
|*3 |   INDEX RANGE SCAN          | EMP_UP_NAME |    1 |      |
| 4 |  TABLE ACCESS BY INDEX ROWID| SALES       |  821 |   34 |
|*5 |   INDEX RANGE SCAN          | SALES_EMP   |   31 |      |
-----------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
  3 - access(UPPER("LAST_NAME") LIKE 'WIN%')
      filter(UPPER("LAST_NAME") LIKE 'WIN%')
  5 - access("E0_"."SUBSIDIARY_ID"="S1_"."SUBSIDIARY_ID"(+)
        AND  "E0_"."EMPLOYEE_ID"  ="S1_"."EMPLOYEE_ID"(+))
```

Database trước tiên lấy result từ table EMPLOYEES thông qua EMP_UP_NAME, sau đó fetch các record tương ứng từ table SALES cho từng employee.

> 💡 Tip
>
> Hiểu ORM của bạn và kiểm soát join.

Nested loops join cho performance tốt nếu driving query trả về một result set nhỏ. Nếu không, optimizer có thể chọn một join algorithm hoàn toàn khác — chẳng hạn hash join được mô tả trong section tiếp theo, nhưng chỉ khi application dùng join để nói cho database biết chính xác cần data nào.

Hash Join

Hash Join

Hash join algorithm nhắm vào điểm yếu của nested loops join: nhiều lần B-tree traversal khi execute inner query. Thay vào đó, nó load các candidate record từ một phía của join vào hash table, để mỗi row từ phía còn lại có thể probe rất nhanh. Tuning hash join đòi hỏi indexing approach hoàn toàn khác nested loops join. Ngoài ra, có thể cải thiện performance của hash join bằng cách select ít column hơn — một thách thức đối với hầu hết ORM tool.

Indexing strategy cho hash join rất khác vì không cần index các join column. Chỉ index cho các independent where predicate mới cải thiện performance của hash join.

> 💡 Tip
>
> Index các independent where predicate để cải thiện performance của hash join.

Xét ví dụ sau. Nó select toàn bộ sale trong sáu tháng vừa qua cùng employee detail tương ứng:

> **TIP**
> Index the *independent* `where` predicates to improve hash join performance.

sider the following example. It selects all sales for the past six months

Filter SALE_DATE là where clause độc lập duy nhất — nghĩa là nó chỉ tham chiếu đến một table và không thuộc join predicate.

Chapter 4: The Join Operation

```
-----------------------------------------------------------------
| Id | Operation          | Name      | Rows  | Bytes | Cost |
-----------------------------------------------------------------
|  0 | SELECT STATEMENT   |           | 49244 |    59M| 12049|
|* 1 |  HASH JOIN         |           | 49244 |    59M| 12049|
|  2 |   TABLE ACCESS FULL| EMPLOYEES | 10000 |     9M|   478|
|* 3 |   TABLE ACCESS FULL| SALES     | 49244 |    10M| 10521|
-----------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - access("S"."SUBSIDIARY_ID"="E"."SUBSIDIARY_ID"
         AND "S"."EMPLOYEE_ID"  ="E"."EMPLOYEE_ID")
   3 - filter("S"."SALE_DATE">TRUNC(SYSDATE@!)
                             -INTERVAL'+00-06' YEAR(2) TO MONTH)
```

The first execution step is a full table scan to load all employees into a

Bước thực thi đầu tiên là full table scan để load toàn bộ employee vào hash table (plan id 2). Hash table dùng join predicate làm key. Ở bước tiếp theo, database lại full table scan table SALES và loại bỏ các sale không thỏa điều kiện SALE_DATE (plan id 3). Với những SALES record còn lại, database access hash table để load employee detail tương ứng.

Mục đích duy nhất của hash table là làm temporary in-memory structure để tránh access table EMPLOYEE nhiều lần. Hash table được load một lần nên không cần index để fetch từng record hiệu

quả. Predicate information xác nhận không có filter nào được áp dụng trên table EMPLOYEES (plan id 2). Query không có independent predicate nào trên table này.

> Important
>
> Indexing join predicate không cải thiện performance của hash join.

Điều đó không có nghĩa là không thể index hash join. Independent predicate có thể được index. Đây là các condition được áp dụng trong một trong hai table access operation. Trong ví dụ trên, đó là filter trên SALE_DATE.

```
CREATE INDEX sales_date ON sales (sale_date);
```

Execution plan sau dùng index này. Tuy nhiên, nó vẫn dùng full table scan cho table EMPLOYEES vì query không có independent where predicate trên EMPLOYEES.

Hash Join

```
-------------------------------------------------------------------
| Id | Operation                     | Name      | Bytes| Cost|
-------------------------------------------------------------------
|  0 | SELECT STATEMENT              |           |   59M| 3252|
|* 1 |  HASH JOIN                    |           |   59M| 3252|
|  2 |   TABLE ACCESS FULL           | EMPLOYEES |    9M|  478|
|  3 |   TABLE ACCESS BY INDEX ROWID| SALES     |   10M| 1724|
|* 4 |    INDEX RANGE SCAN           | SALES_DATE|      |     |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - access("S"."SUBSIDIARY_ID"="E"."SUBSIDIARY_ID"
         AND "S"."EMPLOYEE_ID"  ="E"."EMPLOYEE_ID"  )
   4 - access("S"."SALE_DATE" > TRUNC(SYSDATE@!)
                           -INTERVAL'+00-06' YEAR(2) TO MONTH)
```

Indexing hash join là symmetric — trái ngược với nested loops join. Nghĩa là join order không ảnh hưởng đến indexing. Index SALES_DATE vẫn có thể được dùng để load hash table nếu đảo ngược join order.

> Note
>
> Indexing hash join độc lập với join order.

Một approach khá khác để tối ưu hash join performance là minimize kích thước hash table. Cách này hiệu quả vì hash join tối ưu chỉ có thể xảy ra khi toàn bộ hash table vừa trong memory. Vì vậy optimizer sẽ tự động dùng phía nhỏ hơn của join làm hash table. Oracle execution plan hiển thị memory requirement được estimate trong column “Bytes”. Trong execution plan ở trên, table EMPLOYEES cần chín megabyte nên là phía nhỏ hơn.

Cũng có thể giảm kích thước hash table bằng cách thay đổi SQL query, chẳng hạn thêm condition để database load ít candidate record hơn vào hash table. Tiếp tục ví dụ trên, điều đó có nghĩa thêm filter trên attribute DEPARTMENT để chỉ xét sales staff. Cách này cải thiện hash join performance ngay cả khi không có index trên attribute DEPARTMENT, vì database không cần lưu những employee không thể có sale vào hash table. Khi làm vậy, bạn phải chắc chắn không có SALES record nào thuộc employee không làm việc trong department tương ứng. Hãy dùng constraint để bảo vệ các assumption.

Chapter 4: The Join Operation

Khi minimize kích thước hash table, factor quan trọng không phải số row mà là memory footprint. Thực tế, cũng có thể giảm kích thước hash table bằng cách select ít column hơn — chỉ những attribute thực sự cần:

```
SELECT s.sale_date, s.eur_value
     , e.last_name, e.first_name
  FROM sales s
  JOIN employees e ON (s.subsidiary_id = e.subsidiary_id
                  AND  s.employee_id   = e.employee_id  )
 WHERE s.sale_date > trunc(sysdate) - INTERVAL '6' MONTH
```

Cách này hiếm khi tạo bug vì bỏ nhầm column có lẽ sẽ nhanh chóng dẫn đến error message. Tuy nhiên, vẫn có thể cắt giảm đáng kể kích thước hash table; trong trường hợp này từ 9 megabyte xuống 234 kilobytes — giảm 97%.

```
-------------------------------------------------------------------
| Id | Operation                     | Name      | Bytes| Cost|
-------------------------------------------------------------------
|  0 | SELECT STATEMENT              |           | 2067K| 2202|
|* 1 |  HASH JOIN                    |           | 2067K| 2202|
|  2 |   TABLE ACCESS FULL           | EMPLOYEES |  234K|  478|
|  3 |   TABLE ACCESS BY INDEX ROWID| SALES     |  913K| 1724|
|* 4 |    INDEX RANGE SCAN           | SALES_DATE|      |  133|
-------------------------------------------------------------------
```

> **Tip**
>
> Select ít column hơn để cải thiện hash join performance.

Thoạt nhìn, việc bỏ vài column khỏi SQL statement có vẻ đơn giản, nhưng đây là thách thức thực sự khi dùng object-relational mapping (ORM) tool. Support cho cái gọi là partial object rất hạn chế. Các ví dụ sau cho thấy một số khả năng.

Java

JPA định nghĩa FetchType.LAZY trong annotation @Basic. Nó có thể được áp dụng ở property level:

```
@Column(name="junk")
@Basic(fetch=FetchType.LAZY)
private String junk;
```

Hash Join

JPA provider được tự do ignore nó:

LAZY strategy là một hint cho persistence provider runtime rằng data nên được fetch lazily khi lần đầu được access. Implementation được phép eagerly fetch data mà LAZY strategy hint đã được chỉ định.

—EJB 3.0 JPA, paragraph 9.1.18

Hibernate 3.6 implement lazy property fetching thông qua compile-time bytecode instrumentation. Instrumentation thêm code vào các compiled class và không fetch LAZY property cho đến khi property được access. Approach này hoàn toàn transparent với application, nhưng mở ra một dimension mới của N+1 problem: một select cho mỗi record và property. Điều này đặc biệt nguy hiểm vì JPA không cung cấp runtime control để eager fetch khi cần.

Native query language HQL của Hibernate giải quyết vấn đề bằng clause FETCH ALL PROPERTIES:

```
elect s from Sales s FETCH ALL PROPERTIES
inner join fetch s.employee e FETCH ALL PROPERTIES
where s.saleDate >:dt
```

Clause FETCH ALL PROPERTIES buộc Hibernate eager fetch entity — ngay cả khi dùng instrumented code và LAZY annotation.

Một option khác để chỉ load các column được chọn là dùng data transport object (DTO) thay cho entity. Method này hoạt động giống nhau trong HQL và JPQL, tức là initialize một object ngay trong query:

```
select new SalesHeadDTO(s.saleDate , s.eurValue
                        ,e.firstName, e.lastName)
  from Sales s
  join s.employee e
 where s.saleDate > :dt
```

Query chỉ select data được yêu cầu và trả về object SalesHeadDTO — một Java object đơn giản (POJO), không phải entity.

Chapter 4: The Join Operation

Perl

Framework DBIx::Class không đóng vai trò entity manager nên inheritance không gây ra vấn đề aliasing.[1] Cookbook hỗ trợ approach này. Schema definition sau định nghĩa class Sales ở hai level:

```
package UseTheIndexLuke::Schema::Result::SalesHead;
use base qw/DBIx::Class::Core/;

__PACKAGE__->table('sales');
__PACKAGE__->add_columns(qw/sale_id employee_id subsidiary_id
                            sale_date eur_value/);
__PACKAGE__->set_primary_key(qw/sale_id/);
__PACKAGE__->belongs_to('employee', 'Employees',
          {'foreign.employee_id'   => 'self.employee_id'
          ,'foreign.subsidiary_id' => 'self.subsidiary_id'});

package UseTheIndexLuke::Schema::Result::Sales;
use base qw/UseTheIndexLuke::Schema::Result::SalesHead/;

__PACKAGE__->table('sales');
__PACKAGE__->add_columns(qw/junk/);
```

Class Sales được derived từ class SalesHead và bổ sung attribute còn thiếu. Bạn có thể dùng cả hai class tùy nhu cầu. Lưu ý rằng table setup cũng bắt buộc phải có trong derived class.

Bạn có thể fetch toàn bộ employee detail bằng prefetch hoặc chỉ các column được chọn như dưới đây:

```
my @sales =
   $schema->resultset('SalesHead')
          ->search($cond
                  ,{      join => 'employee'
                   ,'+columns' => ['employee.first_name'
                                   ,'employee.last_name']
                   }
                  );
```

Không thể chỉ load các column được chọn từ root table — trong trường hợp này là SalesHead.

1

http://en.wikipedia.org/wiki/Aliasing_%28computing%29

Hash Join

DBIx::Class 0.08192 generate SQL sau. Nó fetch toàn bộ column từ table SALES và các attribute được chọn từ EMPLOYEES:

```
SELECT me.sale_id,
       me.employee_id,
       me.subsidiary_id,
       me.sale_date,
       me.eur_value,
       employee.first_name,
       employee.last_name
  FROM sales me
  JOIN employees employee
        ON( employee.employee_id   = me.employee_id
       AND  employee.subsidiary_id = me.subsidiary_id)
 WHERE(sale_date > ?)
```

PHP

Version 2 của framework Doctrine hỗ trợ attribute selection tại runtime. Documentation nói rằng partially loaded object có thể behave oddly và yêu cầu dùng keyword partial để acknowledge risk. Ngoài ra, bạn phải select rõ các primary key column:

```
$qb = $em->createQueryBuilder();
$qb->select('partial s.{sale_id, sale_date, eur_value},'
          . 'partial e.{employee_id, subsidiary_id, '
                      . 'first_name , last_name}')
   ->from('Sales', 's')
   ->join('s.employee', 'e')
   ->where("s.sale_date > :dt")
   ->setParameter('dt', $dt, Type::DATETIME);
```

SQL được generate chứa các column được yêu cầu và một lần nữa có SUBSIDIARY_ID cùng EMPLOYEE_ID từ table SALES.

```
SELECT s0_.sale_id       AS sale_id0,
       s0_.sale_date     AS sale_date1,
       s0_.eur_value     AS eur_value2,
       e1_.employee_id   AS employee_id3,
       e1_.subsidiary_id AS subsidiary_id4,
       e1_.first_name    AS first_name5,
       e1_.last_name     AS last_name6,
       s0_.subsidiary_id AS subsidiary_id7,
       s0_.employee_id   AS employee_id8
  FROM sales s0_
 INNER JOIN employees e1_
         ON s0_.subsidiary_id = e1_.subsidiary_id
        AND s0_.employee_id   = e1_.employee_id
 WHERE s0_.sale_date > ?
```

Các object được trả về tương thích với object đã load đầy đủ, nhưng các column thiếu vẫn ở trạng thái chưa initialize. Access chúng không trigger exception.

> Warning
>
> MySQL hoàn toàn không support hash join (feature request #59025).

Chapter 4: The Join Operation

Sort Merge

Sort Merge

Sort-merge join kết hợp hai sorted list giống như một zipper. Cả hai phía của join đều phải được sort theo join predicate.

Sort-merge join cần cùng loại index như hash join, tức là index cho các independent condition để đọc toàn bộ candidate record trong một lần.

Indexing join predicate là vô ích. Cho đến đây, mọi thứ giống hash join. Tuy nhiên, sort-merge join có một đặc tính riêng: symmetry tuyệt đối. Join order không tạo khác biệt — kể cả về performance. Tính chất này đặc biệt hữu ích cho outer join. Với các algorithm khác, hướng của outer join (left hay right) quyết định join order — nhưng không phải với sort-merge join. Sort-merge join thậm chí có thể thực hiện left và right outer join cùng lúc — cái gọi là full outer join.

Dù sort-merge join hoạt động rất tốt khi input đã được sort, nó hiếm khi được dùng vì sort cả hai phía rất tốn kém. Ngược lại, hash join chỉ cần preprocess một phía.

Điểm mạnh của sort-merge join xuất hiện khi input đã được sort sẵn. Có thể đạt điều này bằng cách khai thác thứ tự của index để tránh hoàn toàn các sort operation. Chapter 6, “Sorting and Grouping”, giải thích chi tiết concept này.

Dù vậy, hash join algorithm vẫn vượt trội trong nhiều trường hợp.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Chapter 5

Clustering Data

The Second Power of Indexing

Thuật ngữ cluster được dùng trong nhiều lĩnh vực. Chẳng hạn, star cluster là một nhóm ngôi sao. Computer cluster lại là một nhóm computer làm việc chặt chẽ với nhau — để giải quyết một bài toán phức tạp (high-performance computing cluster) hoặc tăng availability (failover cluster). Nói chung, cluster là những thứ có liên quan và xuất hiện cùng nhau.

Trong computing còn có một loại cluster khác — loại thường bị hiểu sai: data cluster. Clustering data nghĩa là lưu những data được access liên tiếp ở gần nhau để việc access cần ít IO operation hơn. Data cluster rất quan trọng đối với database tuning. Computer cluster cũng rất phổ biến trong ngữ cảnh database — khiến thuật ngữ cluster trở nên mơ hồ. Câu “Let’s use a cluster to improve database performance” là một ví dụ; nó có thể nói đến computer cluster nhưng cũng có thể nghĩa là data cluster. Trong chapter này, cluster nói chung sẽ chỉ data cluster.

Data cluster đơn giản nhất trong SQL database là row. Database lưu tất cả column của một row trong cùng database block nếu có thể. Có ngoại lệ nếu row không vừa trong một block — ví dụ khi có LOB type.

Column Stores

Các database dạng column-oriented, hay column-store, tổ chức table theo column. Model này có lợi khi access nhiều row nhưng chỉ một vài column — một pattern rất phổ biến trong data warehouse (OLAP).

Chapter 5: Clustering Data

Index cho phép cluster data. Cơ sở của điều này đã được giải thích trong Chapter 1, “Anatomy of an Index”: index leaf node lưu indexed column theo thứ tự, nên các value tương tự được lưu cạnh nhau. Điều đó có nghĩa index xây dựng các cluster gồm những row có value tương tự.

Khả năng cluster data quan trọng đến mức tôi gọi nó là power thứ hai của indexing.

Các section sau giải thích cách dùng index để cluster data và cải thiện query performance.

Index Filter Predicates Used Intentionally

Rất thường xuyên, index filter predicate cho thấy index được dùng không đúng do column order sai trong concatenated index. Tuy nhiên, index filter predicate cũng có thể được dùng có chủ đích — không phải để cải thiện range scan performance mà để nhóm những data được access liên tiếp lại với nhau.

Các predicate trong where clause không thể làm access predicate là candidate tốt cho kỹ thuật này:

range" no matter if you index LAST_NAME or UPPER(last_name). This
ion is therefore no good candidate for indexing.

ver the condition on SUBSIDIARY_ID is well suited for indexing. We
even need to add a new index because the SUBSIDIARY_ID is already
ading column in the index for the primary key.

Hãy nhớ rằng LIKE expression có wildcard ở đầu không thể dùng index tree. Nghĩa là indexing LAST_NAME không thu hẹp index range được scan — bất kể bạn index LAST_NAME hay UPPER(last_name). Vì vậy condition này không phải candidate tốt để indexing.

Tuy nhiên, condition trên SUBSIDIARY_ID rất phù hợp để indexing. Chúng ta thậm chí không cần thêm index mới vì SUBSIDIARY_ID đã là leading column trong index của primary key.

Index Filter Predicates Used Intentionally

```
-------------------------------------------------------------------
|Id | Operation                    | Name       | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT             |            |   17 |  230 |
|*1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES  |   17 |  230 |
|*2 |   INDEX RANGE SCAN           | EMPLOYEE_PK|  333 |    2 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter(UPPER("LAST_NAME") LIKE '%INA%')
   2 - access("SUBSIDIARY_ID"=TO_NUMBER(:A))
```

Trong execution plan ở trên, cost value tăng một trăm lần từ INDEX RANGE SCAN đến TABLE ACCESS BY INDEX ROWID tiếp theo. Nói cách khác, table access thực hiện phần lớn công việc. Đây thực sự là pattern phổ biến và bản thân nó không phải vấn đề. Tuy nhiên, nó là contributor đáng kể nhất vào overall execution time của query này.

Table access không nhất thiết là bottleneck nếu các row được access nằm trong một table block, vì database có thể fetch toàn bộ row bằng một read operation. Ngược lại, nếu các row đó nằm rải rác

trong nhiều block, table access có thể trở thành performance problem nghiêm trọng vì database phải fetch nhiều block để lấy toàn bộ row. Nghĩa là performance phụ thuộc vào physical distribution của các row — nói cách khác, phụ thuộc vào clustering của row.

> Note
>
> Mối tương quan giữa index order và table order là một performance benchmark — cái gọi là index clustering factor.

Thực tế có thể cải thiện query performance bằng cách sắp xếp lại row trong table để chúng tương ứng với index order. Tuy nhiên, method này hiếm khi áp dụng được vì bạn chỉ có thể lưu table row theo một sequence. Nghĩa là bạn chỉ có thể optimize table cho một index. Ngay cả khi chọn được một index duy nhất để optimize table, đây vẫn là task khó vì hầu hết database chỉ cung cấp tool rất sơ sài cho việc này. Cái gọi là row sequencing rốt cuộc là một approach khá không thực tế.

Chapter 5: Clustering Data

The Index Clustering Factor

Index clustering factor là indirect measure của xác suất hai index entry liên tiếp tham chiếu đến cùng một table block. Optimizer tính đến xác suất này khi tính cost value của operation TABLE ACCESS BY INDEX ROWID.

Đây chính là nơi power thứ hai của indexing — clustering data — phát huy tác dụng. Bạn có thể thêm nhiều column vào index để chúng tự động được lưu theo một order được xác định rõ. Điều đó biến index thành một tool mạnh nhưng đơn giản để cluster data.

Để áp dụng concept này vào query ở trên, chúng ta phải mở rộng index để cover toàn bộ column trong where clause — kể cả những column không thu hẹp index range được scan:

```
CREATE INDEX empsubupnam ON employees
       (subsidiary_id, UPPER(last_name));
```

The column SUBSIDIARY_ID is the first index column so it can be used as an access predicate. The expression UPPER(last_name) covers the LIKE filter

Column SUBSIDIARY_ID là index column đầu tiên nên có thể được dùng làm access predicate. Expression UPPER(last_name) cover LIKE filter dưới dạng index filter predicate. Index biểu diễn uppercase giúp tiết kiệm một vài CPU cycle trong quá trình execute, nhưng index thông thường trên LAST_NAME cũng hoạt động. Bạn sẽ tìm hiểu thêm về điều này trong section tiếp theo.

```
-------------------------------------------------------------------
|Id | Operation                    | Name       | Rows | Cost |
-------------------------------------------------------------------
| 0 | SELECT STATEMENT             |            |   17 |   20 |
| 1 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES  |   17 |   20 |
|*2 |   INDEX RANGE SCAN           | EMPSUBUPNAM|   17 |    3 |
-------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("SUBSIDIARY_ID"=TO_NUMBER(:A))
       filter(UPPER("LAST_NAME") LIKE '%INA%')
```

Execution plan mới hiển thị chính các operation như trước. Dù vậy, cost value đã giảm đáng kể. Trong predicate information, chúng ta thấy LIKE filter đã được áp dụng ngay trong INDEX RANGE SCAN. Các row không thỏa LIKE filter bị loại ngay lập tức. Table access không còn filter predicate nào nữa. Nghĩa là nó không load những row không thỏa where clause.